CSCI 6461 | TOPIC 4

AArch64 Assembly Programming: Arithmetic, Logic, & Control Flow

ALU mechanics, barrel shifting, bitwise operations,
condition flags, structured branches, and conditional selection

Dr. J. Burns
Computer Systems Architecture — AArch64 Reference Platform

Topic 4: Core Lecture Outline

1. **ALU Operations: Arithmetic, Shifting, & Bitwise Logic**
   - Addition, subtraction, multiplication, division, and barrel shifter integration.
   - Bitwise logic (`and`, `orr`, `eor`, `bic`) and bitfield extraction (`ubfx`).
2. **Condition Flags & Comparison Mechanics**
   - NZCV flag generation, comparison instructions (`cmp`, `tst`), and signed vs. unsigned branches.
3. **Control Flow: Branches & Branchless Selection**
   - Translating loops and `if-else` structures; branch-and-test (`cbz`/`cbnz`).
   - Branch penalties and branchless conditional selection (`csel`, `cset`).

PART 1

ALU Operations: Arithmetic, Shifting, & Bitwise Logic

Arithmetic Primitives: Addition & Subtraction

- Standard AArch64 ALU instructions use a **3-operand non-destructive format**:

```
op  destination,  source1,  source2
```

Instruction	Operation	Flags Updated?
`add x0, x1, x2`	$x_0 \leftarrow x_1 + x_2$	No
`adds x0, x1, x2`	$x_0 \leftarrow x_1 + x_2$	**Yes (Updates NZCV)**
`sub x0, x1, x2`	$x_0 \leftarrow x_1 - x_2$	No
`subs x0, x1, x2`	$x_0 \leftarrow x_1 - x_2$	**Yes (Updates NZCV)**
`neg x0, x1`	$x_0 \leftarrow 0 - x_1$	No (Alias for `sub x0, xzr, x1`)

Immediate Constant Arithmetic

`add x0, x1, #42` adds a 12-bit immediate (0 to 4095). 32-bit registers (`w0`–`w30`) follow identical syntax.

The Barrel Shifter: Free Second-Operand Shifts

- The ALU datapath passes the second source operand through a hardware **barrel shifter** before computation at **zero extra cycle penalty**.

Shift Type	Mnemonic	Operation Meaning	
Logical Shift Left	`lsl #k`	$x \ll k$	(Multiplication by 2^k, zero-fill)
Logical Shift Right	`lsr #k`	$x \gg k$	(Unsigned division by 2^k, zero-fill)
Arithmetic Shift Right	`asr #k`	$x \gg k$	(Signed division by 2^k, sign-fill)

Integrated Shift-Add Examples

`add x0, x1, x2, lsl #3` $\implies x_0 \leftarrow x_1 + (x_2 \times 8)$ (Array indexing by 8-byte doubleword)

`sub x0, x1, x2, lsr #1` $\implies x_0 \leftarrow x_1 - (x_2/2)$ (Subtract half of unsigned value)

Integer Multiplication & Division

- Dedicated hardware units compute 64-bit and 32-bit products and quotients.

Instruction	RTL Semantic Action	Description
`mul x0, x1, x2`	$x_0 \leftarrow x_1 \times x_2$	64-bit integer product (lower 64 bits)
`madd x0, x1, x2, x3`	$x_0 \leftarrow x3 + (x_1 \times x_2)$	Fused Multiply-Accumulate
`msub x0, x1, x2, x3`	$x_0 \leftarrow x_3 - (x_1 \times x_2)$	Fused Multiply-Subtract
`sdiv x0, x1, x2`	$x_0 \leftarrow x_1 / x_2$	Signed division
`udiv x0, x1, x2`	$x_0 \leftarrow x_1 / x_2$	Unsigned division

Division by Zero

AArch64 division by zero does **not** raise an exception; it returns 0 in the destination register.

Bitwise Logic Primitives

- Standard bitwise operations act bit-by-bit across all 64 (or 32) bits in parallel.

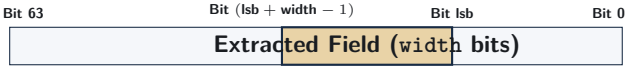
Instruction	Operation	Bitwise RTL Formula
`and x0, x1, x2`	Bitwise AND	$x_0 \leftarrow x_1 \wedge x_2$
`orr x0, x1, x2`	Bitwise OR	$x_0 \leftarrow x_1 \vee x_2$
`eor x0, x1, x2`	Bitwise XOR	$x_0 \leftarrow x_1 \oplus x_2$
`bic x0, x1, x2`	Bit Clear (AND NOT)	$x_0 \leftarrow x_1 \wedge (\sim x_2)$
`mvn x0, x1`	Bitwise NOT	$x_0 \leftarrow \sim x_1$ (Alias: `orn x0, xzr, x1`)

Bit Clear (`bic`) Idiom

`bic x0, x1, x2` sets bits to 0 in x_1 wherever the mask in x_2 has 1s.

Bitfield Extraction: `ubfx`

- Hardware can extract packed fields without explicit manual mask-and-shift steps.
- `ubfx xd, xn, #lsb, #width`: Unsigned bitfield extract.



Example: Extract 8-bit Field Starting at Bit 16

Standard Approach:	`lsr x0, x1, #16`	$\rightarrow$	`and x0, x0, #0xff`	(2 instructions)
Bitfield Extract:	`ubfx x0, x1, #16, #8`			**1 instruction!**

PART 2

Condition Flags & Comparison Mechanics

The NZCV Condition Flags

- Flag bits reside in `pstate` and record status of the most recent flag-setting ALU operation.
- Standard instructions (`add`, `sub`) leave flags unchanged; "`s`" variants update flags.

Flag	Name	Condition Set to 1
N	**Negative**	Result bit 63 is 1 (negative in signed interpretation)
Z	**Zero**	Result is exactly zero
C	**Carry**	Unsigned carry-out (addition) or **no borrow** (subtraction)
V	**Overflow**	Signed arithmetic overflow (result sign is mathematically invalid)

Carry Flag in Subtraction

In AArch64 subtraction ($A - B$), C $= 1$ indicates $A \geq B$ (no borrow required). C $= 0$ indicates $A < B$ (unsigned borrow occurred).

Comparison Instructions: `cmp` and `tst`

- Comparisons evaluate values, set NZCV flags, and discard the computed output.

Instruction	Equivalent Machine Operation	Flags Set
`cmp x0, x1`	`subs xzr, x0, x1`	Sets NZCV based on $x_0 - x_1$
`cmn x0, x1`	`adds xzr, x0, x1`	Sets NZCV based on $x_0 + x_1$ (Compare Negative)
`tst x0, x1`	`ands xzr, x0, x1`	Sets N, Z based on $x_0 \wedge x_1$ (Bit Test)

Checking Flags After `cmp x0, x1`

- If $x_0 == x_1$: **Z flag is set to 1**.
- If $x_0 < x_1$ (signed): $\mathbf{N} \neq \mathbf{V}$.
- If $x_0 < x_1$ (unsigned): $\mathbf{C} = 0$.

Condition Codes: Signed vs. Unsigned Branches

Branch	Meaning	Flag Condition	Comparison Context
`b.eq`	Equal	$Z == 1$	Signed / Unsigned
`b.ne`	Not Equal	$Z == 0$	Signed / Unsigned
`b.lt`	Signed Less Than ($<$)	$N \neq V$	**Signed**
`b.le`	Signed Less or Equal ($\leq$)	$(Z == 1) \vee (N \neq V)$	**Signed**
`b.gt`	Signed Greater Than ($>$)	$(Z == 0) \wedge (N == V)$	**Signed**
`b.ge`	Signed Greater or Equal ($\geq$)	$N == V$	**Signed**
`b.lo`	Unsigned Lower ($<$)	$C == 0$	**Unsigned (Pointers)**
`b.hi`	Unsigned Higher ($>$)	$(C == 1) \wedge (Z == 0)$	**Unsigned (Pointers)**

Takeaway: Always use `b.lo`/`b.hi` for pointer bounds and memory addresses.

PART 3

Control Flow: Branches & Selection

Fused Branch-and-Test: `cbz` and `cbnz`

- **Compare and Branch on Zero (`cbz`) / Non-Zero (`cbnz`):**
  - Tests register contents directly; does **not** alter NZCV flags.
  - Saves 1 instruction compared to an explicit `cmp` + `b.eq`.

Null Pointer Validation

```
cbz x0, .Lhandle_null   // If ptr == NULL, jun
ldr x1, [x0]            // Safe memory access
```

Loop Counter Decrement

```
.Lloop:
subs x1, x1, #1   // count–
cbnz x1, .Lloop   // Loop until count == 0
```

Single Bit Check: `tbz x0, #0, label` branches if bit 0 is zero (even number test).

Translating `if-then-else` Blocks

High-Level C Code

```
if (x > y) {
    result = x - y;
} else {
    result = y - x;
}
```

AArch64 Assembly Mapping

```
// Assume: x0=x, x1=y, x2=result
cmp x0, x1        // Compare x and y
b.le .Lelse       // Invert test: jump if x <= y
sub x2, x0, x1    // Then-body: result = x - y
b .Lend           // Skip else block
.Lelse:
sub x2, x1, x0    // Else-body: result = y - x
.Lend:
```

Assembly Pattern: Invert the high-level condition to fall through into the "then" path.

Loop Translation: Inverted Loop Pattern

High-Level C Loop

```
// Compute array sum
int64_t i = 0;
while (i < n) {
    sum += arr[i];
    i++;
}
```

Optimized AArch64 Loop

```
// x0=arr, x1=n, x2=i, x3=sum
mov x2, xzr                  // i = 0
mov x3, xzr                  // sum = 0
b .Ltest                     // Jump to test first
.Lbody:
ldr x4, [x0, x2, lsl #3]     // Load arr[i]
add x3, x3, x4               // sum += arr[i]
add x2, x2, #1               // i++
.Ltest:
cmp x2, x1                   // i < n?
b.lt .Lbody                  // Loop if true
```

Efficiency Rule: Placing the branch at the bottom requires only **1 branch per iteration**.

Branch Penalties & Branchless Selection (`csel`)

- Pipelined processors speculatively execute instructions ahead of branches.
- An unpredictable branch misprediction wastes 12–20 clock cycles flushing the pipeline.
- **Conditional Select (`csel`):** Evaluates condition flags and moves data in 1 cycle with **0 branch misprediction risk**.

Branchless Maximum: `result` $= (x_1 > x_2)\ ?\ x_1 : x_2$

```
cmp x1, x2              // Set NZCV flags based on x1 - x2
csel x0, x1, x2, gt     // If x1 > x2, x0 = x1; else x0 = x2 (No jumps!)
```

Related Branchless Helpers

`cset w0, eq`	$w_0 \leftarrow (\text{cond}\ ?\ 1 : 0)$	(Convert condition to boolean 1 or 0)
`cinc x0, x1, eq`	$x_0 \leftarrow x_1 + (\text{cond}\ ?\ 1 : 0)$	(Conditional increment)

Review and Self-Test Questions

Topic 4: Comprehensive Summary

- **ALU Integration:** AArch64 embeds a barrel shifter into the second source operand path, giving single-cycle shift-adds (`add x0, x1, x2, lsl #3`).
- **Bitwise Primitives:** Native support for `and`, `orr`, `eor`, and bit clearing via `bic`; field extraction via `ubfx`.
- **Selective Flag Updates:** Only "`s`" variants (`adds`, `subs`) and comparisons (`cmp`, `tst`) update NZCV; carry flag is active ($C = 1$) when subtraction does not borrow.
- **Control Flow Idioms:** Conditional jumps invert high-level conditions; loop inversion cuts branch overhead to 1 branch per iteration.
- **Branch Elimination:** Conditional select instructions (`csel`, `cset`) avoid pipeline misprediction penalties for simple decision paths.

Self-Test Questions: Arithmetic & Logic

1. **Shifted Addition:** What is the exact mathematical operation and resulting decimal value stored in `x0` after executing:

```
mov x1, #12  mov x2, #5  add x0, x1, x2, lsl #2
```

2. **Bit Clear Operation:** If register `x1` contains `0xff` and `x2` contains `0x0f`, what value resides in `x0` after executing `bic x0, x1, x2`?

3. **Signed vs. Unsigned Comparisons:** If `x0` contains -1 (`0xffffffff_ffffffff`) and `x1` contains 1, which branch will be taken after `cmp x0, x1`: `b.lt` or `b.lo`? Why?

4. **Flag Updating:** Explain why `add x0, x1, x2` does not update the Zero (Z) flag even if the resulting sum is exactly zero.

Self-Test Questions: Control Flow & Selection

1. **Loop Inversion Benefit:** Why do optimizing compilers place the conditional loop branch at the bottom of the loop body rather than at the top?
2. **Branchless Translation:** Write an equivalent 2-instruction branchless AArch64 snippet using `cmp` and `csel` to implement:

```
int64_t abs_val = (val < 0) ?  -val :  val;
```

3. **Fused Branch Instructions:** When should an assembly programmer use `cbz x0, label` instead of `cmp x0, #0` followed by `b.eq label`?
4. **Branch Misprediction Trade-off:** Under what conditions will `csel` dramatically outperform a standard branching `if-else` sequence?